40 Days of LeetCode Challenge

40 days, 40 problems
DAY :14

LEETCODE PROBLEM NO. 121 : Best Time To Buy and Sell Stock

Problem Link : https://leetcode.com/problems/contains-duplicate/
Arrays · Greedy Algorithm · Single - Pass Traversal · Conditional Loops · time-space complexity.

STEP BY STEP SOLUTION

Step 1. Understand the question :-

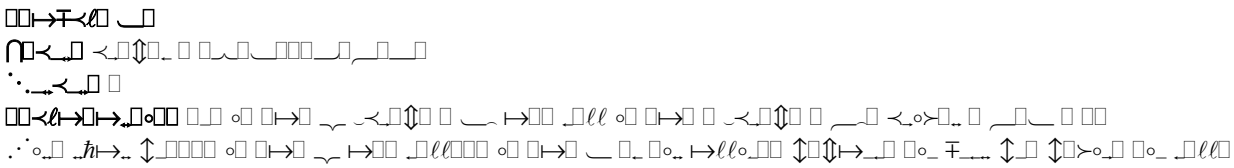
You are given an integer array prices, where prices[i] represents the stock price on the i-th day. You must choose:

- One day to buy a stock.
- A different day in the future to sell that stock.

The objective is to maximize the profit earned from this transaction. Important conditions:

- Buying must happen before selling.
- Only one transaction is allowed.
- If no profitable transaction is possible, return 0.

Step 2. Translating it into simpler language with examples :-





Step 3. Identify the pattern

Imagine monitoring stock prices every day. You need to identify the cheapest price at which you could have bought the stock and determine whether selling it on a later day would generate a better profit.

Instead of checking every possible buying and selling combination, we maintain two variables:

- minPrice: The lowest stock price encountered so far.
- maxProfit: The maximum profit obtained so far.

For every price, we calculate: **{Profit} = {Current Price} -{Minimum Price}**

If the current price is lower than our recorded minimum, we update the minimum buying price. Otherwise, we check whether selling at the current price produces a better profit.

Step 4. Build the logic

Greedy Algorithm — Single-Pass Array Traversal

The Greedy approach works by maintaining the best buying opportunity encountered so far.

At every iteration:

1. We identify whether the current price is a better buying opportunity.
2. If not, we calculate the possible selling profit.
3. We update the maximum profit whenever a better opportunity appears.

This eliminates the need to evaluate every possible pair of days. The solution also uses the concept of a running minimum, where the smallest value encountered during traversal is continuously maintained.

Before implementing the solution, let's understand the variables.

Variable	Purpose	Initial value
minPrice	Stores the minimum stock price encountered	Integer.MAX_VALUE
maxProfit	Stores the maximum profit obtained	0
price	Represents the current stock price	Each array element

Algorithm

1. Initialize minPrice with Integer.MAX_VALUE.
2. Initialize maxProfit with 0.
3. Traverse the prices array using an enhanced for loop.
4. For every current price:
    - If price is smaller than minPrice, update minPrice.
    - Otherwise, calculate the difference between the current price and minimum price.
    - If this difference is greater than maxProfit, update maxProfit.
5. Return maxProfit.

Pseudocode

```
Initialize minPrice = Integer.MAX_VALUE
Initialize maxProfit = 0
For each price in prices:
  If price < minPrice:
    minPrice = price

  Else if price - minPrice > maxProfit:
    maxProfit = price - minPrice

Return maxProfit
```

Step 5. Code in Java

```
class Solution {

  public int maxProfit(int[] prices) {

    int minPrice = Integer.MAX_VALUE;

    int maxProfit = 0;
```

```
    for (int price : prices) {
        if (price < minPrice) {
            minPrice = price;
        } else if (price - minPrice > maxProfit) {
            maxProfit = price - minPrice;   }
    }
    return maxProfit;    }
}
```

How to submit

1. Open LeetCode Problem 121. Best Time to Buy and Sell Stock
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

A. Class declaration

class Solution {

Defines the Solution class required by LeetCode.

B. Method declaration

public int maxProfit(int[] prices) {

- public: Makes the method accessible.
- int: Specifies that the method returns an integer.
- maxProfit: Name of the method.
- int[] prices: Accepts an integer array containing daily stock prices.

C. Initialize minimum price

int minPrice = Integer.MAX_VALUE;

initialize minPrice with Java's built-in constant Integer.MAX_VALUE. It represents the largest possible value of a Java int (2,147,483,647).

NOTE:-

Why use it?

We want the first price encountered to automatically become our minimum price.

For example, if the first price is 7:

7 < 2147483647 → true. Therefore, minPrice becomes 7.

D. Initialize maximum profit

int maxProfit = 0;

We initialize maximum profit to zero because we are allowed to skip the transaction when no profit is possible.

E. Enhanced for loop

for (int price : prices) {

This is an enhanced for loop, also called a for-each loop. It traverses every element of the prices array without requiring an index variable. During each iteration, price stores the current array element.

For example: prices = [7, 1, 5, 3, 6, 4]

The values assigned to price will be: 7 → 1 → 5 → 3 → 6 → 4

F. Check for a new minimum

if (price < minPrice) {

Checks whether the current stock price is lower than the minimum price recorded so far.

G. Update minimum price

minPrice = price;

If the condition is true, we update minPrice with the current price. This means we have discovered a cheaper buying opportunity.

H. Else-if condition

else if (price - minPrice > maxProfit) {

If the current price is not lower than our minimum buying price, we calculate the potential profit. We then check whether this profit exceeds the maximum profit recorded so far. The condition ensures that we only consider profitable improvements.

I. Update maximum profit

maxProfit = price - minPrice;

If the calculated profit is greater than the previous maximum, we update maxProfit.

J. Return the result

return maxProfit;

Returns the maximum profit obtained after traversing the entire array.

Dry Run and Execution Trace

Let's execute the selected solution using: prices = [7, 1, 5, 3, 6, 4];

Initial values: minPrice = Integer.MAX_VALUE ; maxProfit = 0

ITERATION-BY-ITERATION EXECUTION

Iteration	Current price	Condition / Action	minPrice	maxProfit
1	7	New minimum	7	0
2	1	New minimum	1	0
3	5	Profit = 5 − 1	1	4
4	3	Profit = 3 − 1	1	4
5	6	Profit = 6 − 1	1	5
6	4	Profit = 4 − 1	1	5

Final output = 5 I.e buy at 1 and sell at 6.

Understanding the decreasing-price example

prices = [7, 6, 4, 3, 1];

Current price	Minimum price	Maximum profit
7	7	0
6	6	0
4	4	0
3	3	0
1	1	0

Since every new price becomes the minimum, the profit condition never produces a positive result.

Output = 0

Step 7. Time complexity & Auxiliary space complexity

Complexity	Result	Explanation
Time Complexity	O(n)	Every element is visited exactly once.
Auxiliary Space	O(1)	Only two variables are maintained.

NOTE:-

Why is this efficient?

The brute-force approach checks every possible buying and selling pair, resulting in $O(n^2)$ time complexity. Our Greedy approach maintains the minimum buying price and calculates potential profit during a single traversal. Therefore, it achieves O(n) time and O(1) auxiliary space, which is asymptotically optimal for this problem

SCREENSHOT:-

